



Instituto Politécnico
de Viana do Castelo

LICENCIATURA EM ENGENHARIA INFORMÁTICA

OPÇÃO II – APRENDIZAGEM ORGANIZACIONAL / SISTEMAS DE SUPORTE À DECISÃO

CP3 – AI Music Composer

N.º 24611 – Diogo Fontes – diogofontes@ipvc.pt

N.º 31388 – Simão Mendes – simaomendes@ipvc.pt

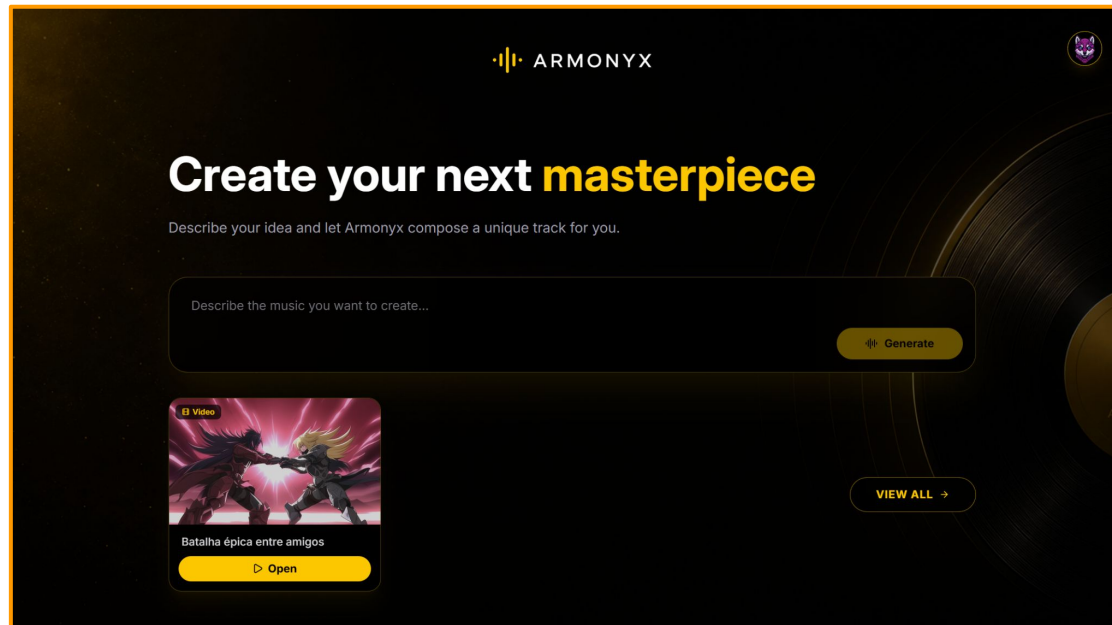
Prof. Jorge Ribeiro – jribeiro@estg.ipvc.pt

Prof. Abel Dantas – adantas@estg.ipvc.pt

Índice

1.	Introdução	Diap.3
2.	Objetivos	Diap.4
3.	Contexto	Diap.5
4.	Arquitetura	Diap.6
5.	Base de Dados	Diap.7
6.	Tecnologias Utilizadas	Diap.8
7.	Justificação das Tecnologias	Diap.9
8.	Limitações e Dificuldades	Diap.10
9.	Demonstração	Diap.11
10.	Conclusão	Diap.12
11.	Referências	Diap.13

■ Introdução



O *Armonyx* é uma plataforma de inteligência artificial capaz de gerar *Anime Music Videos* (AMVs) completos a partir de descrições em linguagem natural, demonstrando a orquestração de *LLMs* e modelos generativos de áudio e imagem numa solução integrada de ponta a ponta.

■ Objetivos

Os objetivos do trabalho foram os seguintes:

- ❖ Gerar música original com *MusicGen* a partir de prompts descritivos em linguagem natural;
- ❖ Gerar imagens de cenas com *Stable Diffusion*, coerentes com o tema do *AMV*;
- ❖ Renderizar o vídeo final (imagens + música + transições) com *FFmpeg*;
- ❖ Orquestrar todo o processo com um agente conversacional *Qwen2.5:14b* (classificação de intenção + plano criativo);

■ Contexto

O ponto de partida deste projeto foi uma proposta de *AI Music Composer*, um sistema capaz de gerar música original a partir de uma descrição textual, em que o utilizador indica um estilo ou *mood* e o sistema produz a faixa correspondente.

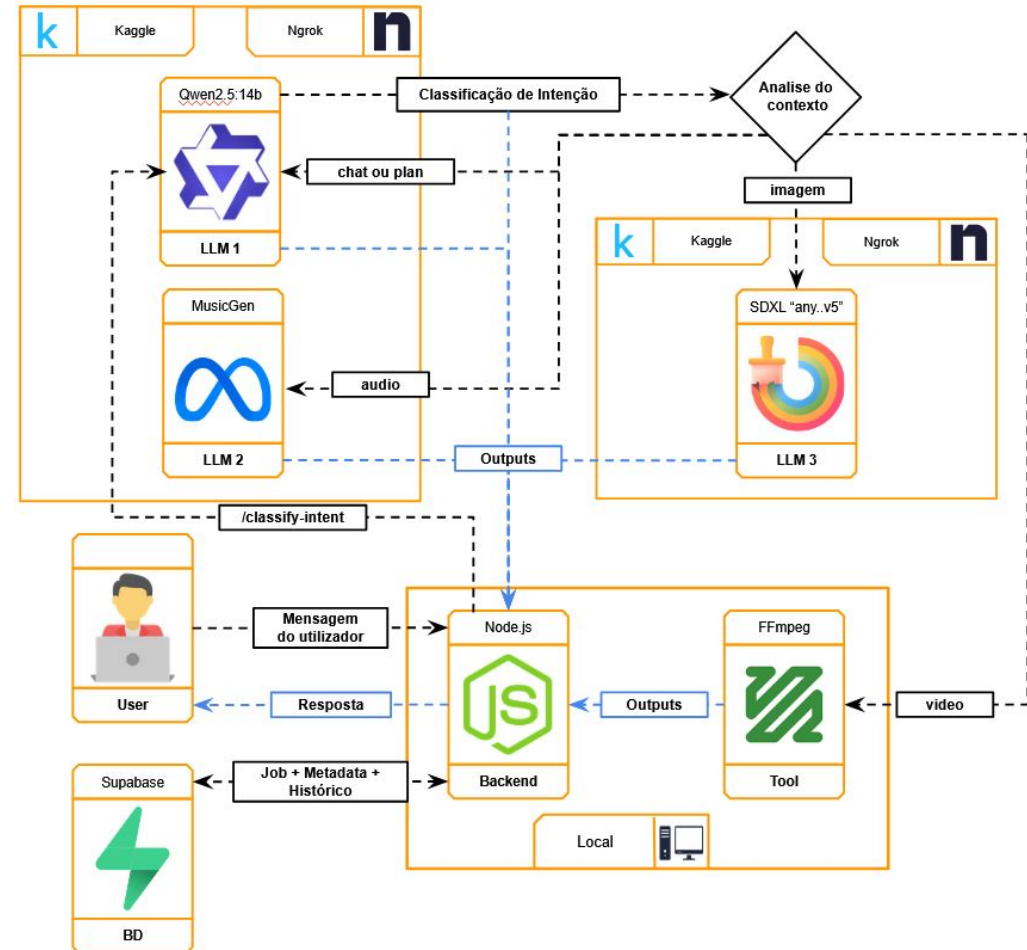
A proposta foi estendida para um sistema completo de criação de *Anime Music Videos (AMVs)*. Para além da geração de música, foi integrado um planeador criativo que gera um *storyboard* coerente com o tema definido pelo utilizador, um gerador de imagens por difusão que produz automaticamente as cenas descritas nesse *storyboard*, e um motor de renderização com *FFmpeg* que sincroniza música e imagens num vídeo final com transições.

O resultado é uma plataforma *open-source* de ponta a ponta, sem qualquer dependência de *APIs* pagas, com os modelos executados remotamente via *Kaggle*.

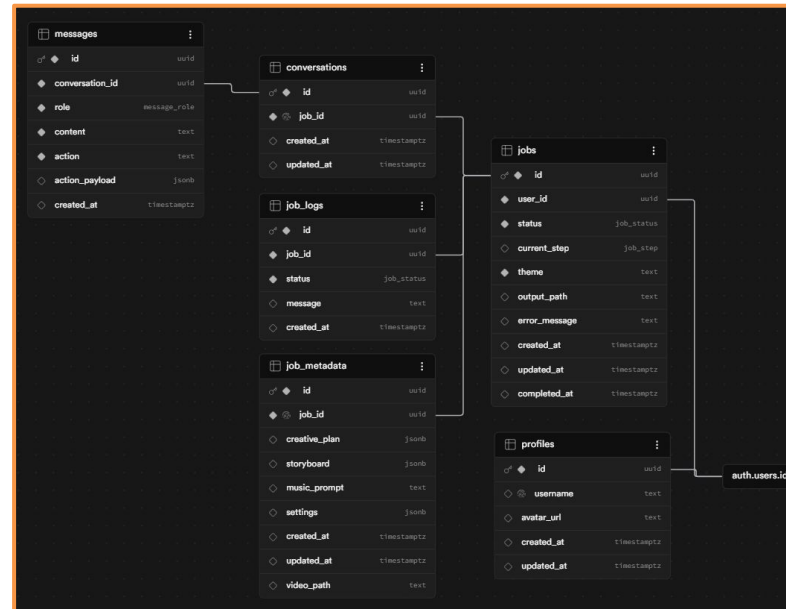
■ Arquitetura

O *Backend* (*Node.js/Express*) orchestra todo o *pipeline*, guiado pelo *Qwen2.5:14b* (*Kaggle* via *Ngrok*), que classifica a intenção do utilizador e gera o plano criativo.

Com base nesse plano, o *MusicGen* e o *Stable Diffusion XL* geram áudio e imagens, que o *FFmpeg* combina num vídeo final. Todo o estado é persistido no *Supabase* e refletido em tempo real no *Frontend* em *React*.



■ Base de Dados



A base de dados centra-se na tabela *jobs*, que representa cada sessão de criação de um AMV e armazena o seu estado, tema e caminhos de ficheiros gerados.

A cada *job* está associada uma conversation com as respetivas *messages*, que registam o histórico do diálogo entre o utilizador e o assistente. Os *metadados* são persistidos em *job_metadata*, enquanto os *job_logs* guardam o registo cronológico de cada fase do *pipeline*. Por fim, a tabela *profiles* estende o sistema de autenticação do *Supabase* com informação de perfil do utilizador.

■ Tecnologias Utilizadas

- ❖ **Qwen2.5:14b**: Classificação de intenção e plano criativo;
- ❖ **MusicGen**: Geração de música original;
- ❖ **Stable Diffusion XL**: Geração das imagens do *storyboard*;
- ❖ **FFmpeg**: Renderização do vídeo final;
- ❖ **Node.js / Express**: *Backend* e orquestração do *pipeline*;
- ❖ **Supabase**: Base de dados, autenticação e armazenamento;
- ❖ **React**: Interface do utilizador em tempo real;
- ❖ **Kaggle**: *GPU* gratuita para correr os modelos;
- ❖ **Ngrok**: Expõe os *workers* do *Kaggle* ao *backend*;
- ❖ **Tailwind CSS**: Estilização da interface;



ngrok



Tailwind CSS



kaggle



■ Justificação das tecnologias

Sem custos de API/open-source:

- ❖ *Qwen2.5:14b, MusicGen, Stable Diffusion XL*: modelos *open-source*, sem subscrições nem limites de uso;
- ❖ *Kaggle*: *GPU* gratuita suficiente para correr os 3 modelos em paralelo;

Adequação ao caso de uso:

- ❖ *SDXL "anything-v5"*: afinado para estilo anime, alinhado com o tema dos *AMVs*;
- ❖ *FFmpeg*: transições profissionais e sincronização áudio/vídeo sem custos de licença;
- ❖ *Ngrok*: forma simples de expor os *workers Kaggle* ao *backend* sem *deploy* em *cloud*;

Produtividade de desenvolvimento:

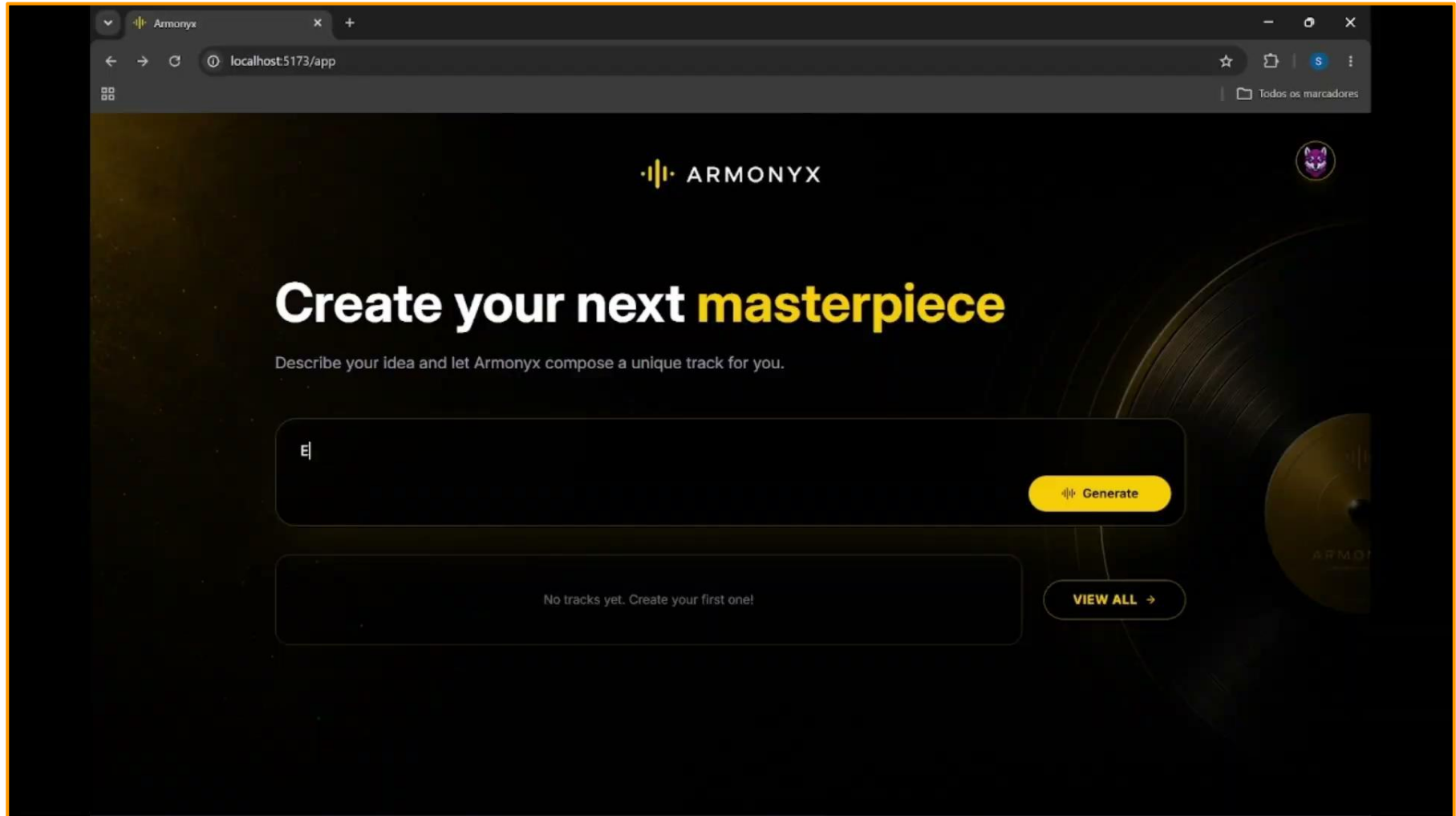
- ❖ *Node.js/Express*: assíncrono por natureza, ideal para orquestrar jobs longos via *polling*;
- ❖ *Supabase*: *BD* + autenticação + *realtime* prontos, sem montar infraestrutura própria;
- ❖ *React* + *Tailwind*: *UI* reativa construída rapidamente, com atualização de progresso em tempo real;

■ Limitações e Dificuldades

Algumas limitações que tivemos:

- ❖ Vídeo final via *FFmpeg* (*slideshow* com transições), não vídeo fluido, modelos de geração de vídeo exigem *GPU* muito acima do disponível gratuitamente;
- ❖ Duração do áudio limitada a 30s, *MusicGen* falha de forma reprodutível acima de ~40s (fora do treino);
- ❖ Apenas uma geração por job em simultâneo, pedidos extra ficam em espera;
- ❖ Regeneração "tudo ou nada", não é possível recriar só uma cena, só o conjunto todo;
- ❖ Duas *notebooks Kaggle* em paralelo (*Qwen+MusicGen* / *SDXL*), que duplica complexidade;

■ Demonstração



■ Conclusão

O *Armonyx* cumpre os objetivos propostos, orquestrando com sucesso o *Qwen2.5:14b*, o *MusicGen*, o *Stable Diffusion XL* e o *FFmpeg* numa *pipeline end-to-end* que transforma uma simples descrição em linguagem natural num *AMV* completo. O projeto demonstra que é possível construir uma plataforma criativa funcional e coesa recorrendo exclusivamente a modelos *open-source*, sem qualquer dependência de *APIs* pagas.

A arquitetura distribuída, *backend* local, *workers* executados no *Kaggle* e expostos via *Ngrok*, com persistência no *Supabase*, revela-se viável mesmo com recursos de *hardware* limitados. As limitações identificadas, como a duração máxima do áudio, a renderização do vídeo em formato de *slideshow* e a regeneração tudo-ou-nada, não comprometem a validação do conceito e definem antes um roteiro claro para trabalho futuro.

■ Referências

- ❖ React Router com Tailwind: <https://tailwindcss.com/docs/installation/framework-guides/react-router>
- ❖ Ngrok: <https://ngrok.com/docs/start>
- ❖ Kaggle: <https://www.kaggle.com/>
- ❖ Supabase: <https://supabase.com/docs>
- ❖ MusicGen: <https://huggingface.co/facebook/musicgen-medium>
- ❖ Qwen: <https://huggingface.co/Qwen/Qwen2.5-14B-Instruct>
- ❖ Stable Diffusion XL: <https://huggingface.co/stablediffusionapi/anything-v5>
- ❖ FFmpeg: <https://ffmpeg.org/documentation.html>
- ❖ Claude: <https://claude.ai/>

o teu • de partida



www.ipvc.pt